```
!pip install gensim
```

```
Collecting gensim
  Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (8.4 kB)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
Downloading gensim-4.4.0-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (27.9 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.9/27.9 MB 62.0 MB/s eta 0:00:00
Installing collected packages: gensim
Successfully installed gensim-4.4.0
```

STEP 2 — Import Libraries

```
import gensim.downloader as api   # load pre-trained embeddings
import numpy as np               # numerical operations
import matplotlib.pyplot as plt  # plotting
from sklearn.manifold import TSNE # t-SNE dimensionality reduction
```

STEP 3 — Load Pre-trained Embeddings

```
model = api.load("glove-wiki-gigaword-50")
```

```
[==================================================] 100.0% 66.0/66.0MB downloaded
```

```
print("Vocabulary size:", len(model))
print("Vector dimension:", model.vector_size)
print("Example vector (king):")
print(model['king'])
```

```
Vocabulary size: 400000
Vector dimension: 50
Example vector (king):
[ 0.50451   0.68607  -0.59517  -0.022801  0.60046  -0.13498  -0.08813
  0.47377  -0.61798  -0.31012  -0.076666  1.493    -0.034189 -0.98173
  0.68229   0.81722  -0.51874  -0.31503  -0.55809   0.66421   0.1961
 -0.13495  -0.11476  -0.30344   0.41177  -2.223    -1.0756   -1.0783
 -0.34354   0.33505   1.9927   -0.04234  -0.64319   0.71125   0.49159
  0.16754   0.34344  -0.25663  -0.8523    0.1661    0.40102   1.1685
 -1.0137   -0.21585  -0.15155   0.78321  -0.91241  -1.6106   -0.64426
 -0.51042 ]
```

STEP 4 — Select 30–50 Words

```
words = [
    # Royalty
    'king','queen','prince','princess',

    # Education
    'school','college','university','teacher','student',

    # Medical
    'doctor','nurse','hospital','patient','medicine',

    # Geography
    'india','china','france','paris','delhi','london',

    # Technology
    'computer','internet','software','hardware','phone',

    # Transport
    'car','bus','train','airplane','road',

    # Entertainment
    'music','dance','movie','song','actor'
]
```

```
vectors = np.array([model[word] for word in words])
```

STEP 5 — Apply t-SNE (CORE STEP)

```
tsne = TSNE(
    n_components=2,
    perplexity=10,
    random_state=42,
    n_iter=1000
)

vectors_2d = tsne.fit_transform(vectors)
```

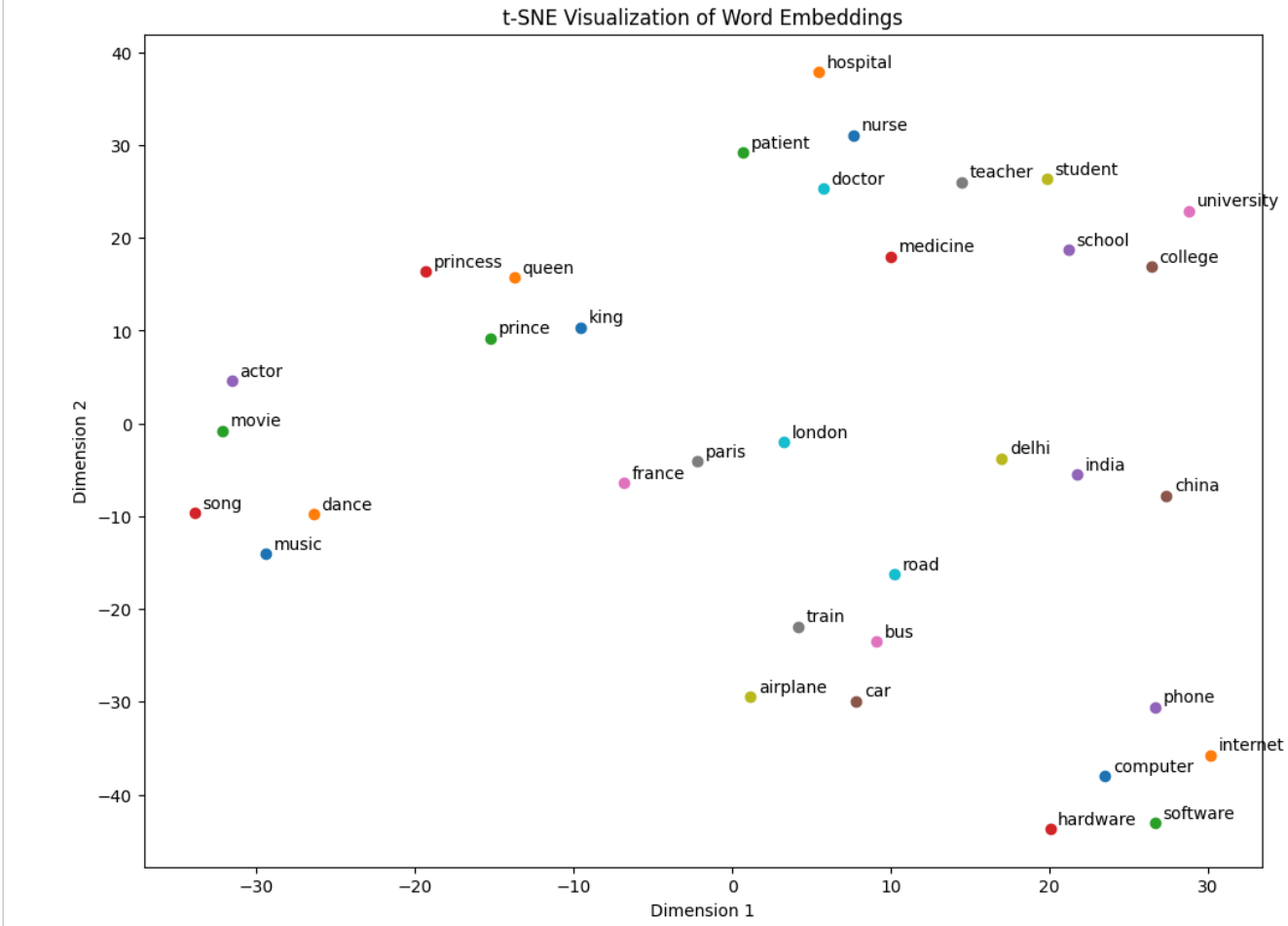
```
/usr/local/lib/python3.12/dist-packages/sklearn/manifold/_t_sne.py:1164: FutureWarning: 'n_iter' was renamed to 'max_iter' i
  warnings.warn(
```

STEP 6 — Plot Visualization

```
plt.figure(figsize=(12, 9))

for i, word in enumerate(words):
    plt.scatter(vectors_2d[i, 0], vectors_2d[i, 1])
    plt.text(vectors_2d[i, 0]+0.5, vectors_2d[i, 1]+0.5, word)

plt.title("t-SNE Visualization of Word Embeddings")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.show()
```



STEP 7 — Interpretation (8–10 sentences)

The t-SNE visualization shows clear clustering of semantically related words. Words related to education such as school, college, and university appear close together, forming a meaningful cluster. Medical-related words like doctor, nurse, and hospital are also grouped together. Geographic locations such as India, China, and France appear near each other, indicating regional similarity.

Technology-related terms like computer, software, and internet form another cluster. Entertainment-related words such as music, song, and movie are placed nearby. These clusters demonstrate that word embeddings capture semantic relationships effectively.

Some words may appear closer than expected due to overlapping contexts in the training data. Overall, t-SNE helps visually understand how embeddings organize word meanings in vector space.